

Android 应用 安全分析报告

RealPersonAuthActivity 模块

报告编号: SEC-2026-0515-001

风险评级: 严重 (Critical)

报告日期: 2026-05-15

仅用于安全研究与教学, 禁止用于任何非法用途
由 Kiro 协助整理

Android 应用安全分析报告

报告编号: SEC-2026-0515-001 **报告日期:** 2026-05-15 **目标模块:** RealPersonAuthActivity (真人认证) **机密等级:** 内部研究 / 教学示例 **作者:** [研究员代号] **声明:** 本报告仅用于安全研究与教学, 禁止用于任何非法用途

一、执行摘要 (Executive Summary)

测试目标: 对一款国内直播/社交类 Android 应用 `com.phone.secondmove.liveproject` 进行黑盒安全测试, 重点评估「真人认证」模块的安全性。

核心结论: - 共发现 **5 个严重漏洞、4 个高危漏洞、4 个中危漏洞、3 个低危问题** - **真人认证功能可被完全绕过**, 可能导致大规模虚假认证 - 应用未做任何加固和混淆, 逆向门槛极低 - 涉及人脸数据, 存在《个人信息保护法》合规风险

风险评级: 严重 (Critical)

修复建议优先级: 建议 **7 天内紧急修复 P0 问题**, 30 天内完成全部修复。

二、测试范围与方法

2.1 测试范围

项目	说明
测试包	<code>base.apk</code> (SHA256: **)
测试模块	真人认证 (RealPersonAuthActivity)
测试设备	Pixel 6 (Android 13) / 雷电模拟器 9.0
测试时间	2026-05-10 至 2026-05-15

2.2 测试方法

- 静态分析:** JADX 4.x、apktool 2.9、MobSF 3.9

- **动态分析**: Frida 16.x、objection 1.11、Xposed
- **流量分析**: HTTP Toolkit、Charles 4.6、mitmproxy 10
- **测试标准**: OWASP MASVS v2.0 / OWASP MSTG / GB/T 35273

2.3 测试边界声明

本测试在受控环境下进行，**未对生产环境造成任何影响**。所有测试账号为本人注册，未访问其他用户数据。测试发现的漏洞**不会公开 PoC**，仅向厂商提交。

三、应用基本信息

3.1 包信息

PackageName: com.phone.secondmoveliveproject
VersionName: X.X.X
VersionCode: XXX
MinSdk: 21
TargetSdk: 30
Signed by: CN=XXX (未通过 Google Play 签名校验)

3.2 技术栈

类别	组件
网络框架	EasyHttp (基于 RxJava + Retrofit + OkHttp)
图片选择	Matisse 0.5.3
图片压缩	Luban 1.1.8
图片加载	Glide 4.x
直播 SDK	待进一步分析
推送 SDK	待进一步分析

3.3 安全配置审计

项目	状态	风险
代码混淆 (ProGuard/R8)	未启用	高
加固保护	未加固	高
反调试	无	中
Root 检测	无	低
SSL Pinning	未实现	高
Network Security Config	默认配置	中
<code>android:debuggable</code>	false	正常
<code>android:allowBackup</code>	true	中

四、漏洞清单总览

编号	漏洞名称	等级	模块	状态
VUL-001	真人认证可被相册图片绕过	严重	RealPersonAuthActivity	待修复
VUL-002	上传接口缺乏签名校验	严重	EasyHttp 全局	待修复
VUL-003	未实现 SSL Pinning	严重	网络层	待修复
VUL-004	客户端结果可被 Hook 篡改	严重	RealPersonAuthActivity	待修复
VUL-005	Release 包泄露调试日志	严重	全局	待修复
VUL-006	FileProvider 配置疑似过宽	高危	AndroidManifest	待复验
VUL-007	未做文件类型/大小校验	高危	uploadPic / uploadAvatar	待修复
VUL-008	应用未做混淆与加固	高危	整体	待修复
VUL-009	Activity 泄漏 + 回调劫持风险	高危	RealPersonAuthActivity	待修复
VUL-010	权限请求不完整（缺相机）	中危	picImage	待修复
VUL-011	picUrl 成员变量 TOCTOU	中危	RealPersonAuthActivity	待修复
VUL-012	服务端错误信息直接外显	中危	onSuccess 分支	待修复
VUL-013	缓存图片未清理	中危	Luban 输出	待修复
VUL-014	硬编码 requestCode	低	RealPersonAuthActivity	建议优化
VUL-015	日志 Tag 个人化 (zq)	低	全局	建议规范
VUL-016	未做代码加固	低	整体	见 VUL-008

五、漏洞详情

VUL-001：真人认证可被相册图片绕过

- **风险等级：**严重 (Critical)
- **CVSS 3.1：**8.6 (AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N)
- **CWE：**CWE-287 (Improper Authentication)
- **OWASP Mobile Top 10：**M4 - Insufficient Authentication / Authorization

漏洞位置

```
com.phone.secondmoveliveproject.activity.mine.RealPersonAuthActivity  
方法: picImage()
```

漏洞描述

真人认证功能调用 Matisse 图片选择器时，未强制使用相机拍摄模式，允许用户从相册选择任意图片提交认证。结合服务端可能未做活体检测，攻击者可使用网络下载的人脸照片完成虚假认证。

漏洞代码

```
Matisse.from(this)  
    .choose(MimeType.ofImage(), false)  
    .capture(true)           // 此处声明允许拍照  
    .captureStrategy(...)  
    .capture(false)         // 但被此处覆盖为 false  
    .maxSelectable(1)  
    .forResult(REQUEST_CODE_CHOOSE_Head);
```

复现步骤

1. 使用 Frida 启动应用
2. 进入「我的」 -> 「真人认证」
3. 点击上传按钮
4. 在弹出的选择器中选择相册
5. 选择任意一张人像照片
6. 点击提交，观察认证结果

影响范围

- 影响所有用户的认证可信度
- 可被恶意用户用于：批量注册虚假认证账号、冒充他人、规避平台监管

业务影响

- 平台信任体系崩塌
- 监管处罚风险（涉及《互联网直播服务管理规定》）
- 黑产可批量产出"已认证账号"用于诈骗

修复建议

1. **短期**：移除 `capture(false)` 调用，强制使用相机
2. **长期**：接入正规活体检测 SDK（推荐：阿里云人脸识别、腾讯云慧眼、商汤、Face++）
3. **服务端**：对图片进行 EXIF 校验、AI 鉴伪、人脸比对

修复代码示例

```
// 不再使用 Matisse 选图，改为直接调起活体检测
ZIMFacade zimFacade = ZIMFacadeBuilder.create(this);
zimFacade.verify(zimId, true, new ZIMCallback() {
    @Override
    public boolean response(ZIMResponse response) {
        if (response.code == 1000) {
            uploadPic(...); // 活体检测通过
        }
        return true;
    }
});
```

VUL-002：上传接口缺乏签名校验

- **风险等级**：严重 (Critical)
- **CVSS 3.1**：9.1 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
- **CWE**：CWE-345 (Insufficient Verification of Data Authenticity)

漏洞描述

图片上传接口 `APP_UPLOAD` 在抓包分析中未发现请求签名 (sign)、时间戳 (timestamp)、随机数 (nonce) 等防重放机制。攻击者抓取一次合法请求后，可通过脚本无限重放或伪造任意上传请求。

抓包证据（脱敏）

```
POST /api/upload HTTP/1.1
Host: api.xxx.com
Content-Type: multipart/form-data; boundary=xxx
Content-Length: xxx
Authorization: Bearer eyJxxxxx          (仅有 Token, 无签名)

-----XXX
Content-Disposition: form-data; name="files"; filename="xxx.jpg"
Content-Type: image/jpeg

[binary data]
-----XXX--
```

PoC（脱敏，仅展示原理）

```
# 仅用于演示原理，不提供可直接运行的攻击代码
import requests
token = "..." # 通过正常登录获取
files = {'files': open('any_face.jpg', 'rb')}
headers = {'Authorization': f'Bearer {token}'}
# 攻击者可循环调用此请求批量上传，无任何阻断机制
```

修复建议

- 加入签名机制： `sign = HMAC-SHA256(appSecret, timestamp + nonce + body)`
- 服务端校验时间戳偏移（±5分钟）
- 服务端记录 nonce 防重放（10分钟内不允许重复）
- 引入设备指纹（含设备ID + 应用包名 + 签名Hash）
- 关键操作引入风控（人机校验、行为分析）

VUL-003：未实现 SSL Pinning

- **风险等级**：严重 (Critical)
- **CVSS 3.1**：7.4 (AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N)
- **CWE**：CWE-295 (Improper Certificate Validation)

漏洞描述

应用网络层未实现证书锁定（Certificate Pinning），攻击者可通过： - 用户安装根证书 + 中间人代理（Charles / Burp / mitmproxy） - 公共 Wi-Fi 部署 SSL 剥离 - 运营商劫持

…等方式截获并篡改客户端与服务端之间的通信。鉴于本模块涉及人脸图像与身份证照片，**信息泄露风险极高**。

修复建议

```
// OkHttp 证书锁定示例
CertificatePinner pinner = new CertificatePinner.Builder()
    .add("api.xxx.com", "sha256/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=")
    .add("api.xxx.com", "sha256/BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB") // 备用证书
    .build();

OkHttpClient client = new OkHttpClient.Builder()
    .certificatePinner(pinner)
    .build();
```

并配置 `network_security_config.xml`：

```
<network-security-config>
  <domain-config>
    <domain includeSubdomains="true">api.xxx.com</domain>
    <pin-set>
      <pin digest="SHA-256">AAAA...</pin>
      <pin digest="SHA-256">BBBB...</pin>
    </pin-set>
    <trust-anchors>
      <certificates src="system" />
    </trust-anchors>
  </domain-config>
</network-security-config>
```

VUL-004：客户端认证结果可被 Hook 篡改

- **风险等级**：严重
- **CWE**：CWE-602 (Client-Side Enforcement of Server-Side Security)

漏洞描述

`onSuccess` 中直接信任服务端返回的 JSON 中的 `data[0].name` 字段作为图片地址，并立即调用 `realPersonAuth()`。该过程**完全在客户端进行**，攻击者可通过 Frida hook `optString("name")` 的返回值，**替换成预先认证通过的图片 URL**，绕过认证。

漏洞代码

```
JSONArray data = jsonObject.optJSONArray("data");
if (data.length() > 0) {
    this.picUrl = data.optJSONObject(0).optString("name");
    this.realPersonAuth(); // 直接进入认证
}
```

Hook 演示（脱敏）

```
Java.perform(function() {
    var JSONObject = Java.use('org.json.JSONObject');
    JSONObject.optString.overload('java.lang.String').implementation = function(key) {
        var ret = this.optString(key);
        if (key === "name" && ret.indexOf(".jpg") !== -1) {
            // 攻击者可在此替换为任意已认证图片URL
            return "https://cdn.xxx.com/legit_face.jpg";
        }
        return ret;
    };
});
```

修复建议

- 关键认证流程**全部在服务端完成**
- 客户端只负责上传，认证状态由服务端通过下行长连接 / 主动查询返回
- 引入服务端环境感知（IP 频率、设备指纹、行为分析）

VUL-005: Release 包泄露调试日志

- **风险等级**: 严重
- **CWE**: CWE-532 (Insertion of Sensitive Information into Log File)

漏洞描述

代码中存在大量未关闭的 `Log.e` 调用，且 Tag 使用开发者花名缩写 `zq`，明显是调试代码遗留。

```
Log.e("zq", "开始压缩");
Log.e("zq", "压缩成功, 压缩后图片位置:" + file.getPath() + "压缩后图片大小:" + (file.length() / 1024) + "KB");
Log.e("===Image=jsonObject==", ((int) ((j * 100) / j2)) + "===");
```

风险

- Release 包如未关闭 Log，文件路径暴露在 logcat 中
- 第三方 SDK（友盟、Bugly 等）可能自动收集 Log，导致用户照片路径外泄
- 在 Android 4.x / Root 设备上，其他 App 可读取 logcat

修复建议

```
// 统一 LogUtil
public class L {
    private static final boolean DEBUG = BuildConfig.DEBUG;
    public static void e(String tag, String msg) {
        if (DEBUG) Log.e(tag, msg);
    }
}
```

并在 `proguard-rules.pro` 中：

```
-assumenosideeffects class android.util.Log {
    public static *** d(...);
    public static *** v(...);
    public static *** i(...);
    public static *** w(...);
    public static *** e(...);
}
```

VUL-006: FileProvider 配置疑似过宽

- **风险等级**：高危
- **CWE**：CWE-200 (Information Exposure)

漏洞描述

```
.captureStrategy(new CaptureStrategy(true, BaseConstants.APP_FileProvider, "test"))
```

需要审查 `AndroidManifest.xml` 中 `FileProvider` 的 `<paths>` 配置。常见错误模式：

```
<external-path name="all" path="." />      <!-- 暴露整个外置存储 -->
<external-files-path name="all" path="." /> <!-- 暴露 App 沙盒 -->
```

修复建议

收紧 `paths` 配置，只暴露必要的子目录：

```
<paths>
  <cache-path name="capture" path="capture/" />
</paths>
```

VUL-007：未做文件类型 / 大小校验

- **风险等级**：高危
- **CWE**：CWE-434 (Unrestricted Upload of File with Dangerous Type)

漏洞描述

```
httpParams.put("files", file, file.getName(), uIProgressResponseCallBack);
```

- 未校验真实 MIME（仅靠扩展名）
- 未限制文件大小（Luban `ignoreBy(100)` 只是不压缩，不是限制大小）
- `file.getName()` 直接使用用户文件名，可能含 `../` 路径穿越字符

修复建议

```
// 1. 校验文件大小
if (file.length() > 5 * 1024 * 1024) {
    return; // 拒绝大于 5MB
}

// 2. 校验 MIME
String mime = MetadataRetriever.getMimeType(file);
if (!"image/jpeg".equals(mime) && !"image/png".equals(mime)) {
    return;
}

// 3. 文件名重命名
String safeName = UUID.randomUUID().toString() + ".jpg";
httpParams.put("files", file, safeName, callback);
```

VUL-008：应用未做混淆与加固

- **风险等级**：高危
- **CWE**：CWE-656 (Reliance on Security Through Obscurity)

漏洞描述

JADX 反编译输出可读性极高： - 类名 `RealPersonAuthActivity` 完整保留 - 字段名 `picUrl` , `picUrl1` , `headPath` 清晰 - 业务方法名 `uploadPic` , `uploadAvatar` , `realPersonAuth` 自解释

说明应用： - 未启用 ProGuard / R8 混淆 - 未使用任何加固方案

修复建议

1. **启用 R8 混淆**： `build.gradle` 中 `minifyEnabled true`
2. **采用商业加固**：腾讯乐固、360 加固、爱加密、梆梆
3. **关键算法 Native 化**：签名/加密迁移到 .so 并配合 OLLVM
4. **加入完整性校验**：检测 dex / .so / 签名是否被篡改

VUL-009：Activity 泄漏 + 回调劫持风险

- **风险等级**：高危
- **CWE**：CWE-401 (Memory Leak), CWE-362 (Race Condition)

漏洞描述

```
final String pathFromUri = FileUtil.getPathFromUri(intent.getData());
Luban.with(this).load(...).setCompressListener(new OnCompressListener() {
    public void onSuccess(File file) {
        RealPersonAuthActivity.this.uploadPic(file);
    }
}).launch();
```

- 匿名内部类持有外部 Activity 引用 -> 内存泄漏
- 用户快速退出时，回调到来仍会操作已销毁 Activity
- 没有 `isDestroyed()` 校验，存在 Race Condition

修复建议

```
public void onSuccess(File file) {
    if (isDestroyed() || isFinishing()) return;
    uploadPic(file);
}
```

或改用 `WeakReference`、Lifecycle-aware 组件、Kotlin 协程 + `viewModelScope`。

VUL-010 ~ VUL-013（中危问题）

编号	名称	简述	修复建议
VUL-010	权限请求不完整	<code>requestStoragePermission()</code> 只检查存储，未请求相机权限	调起相机前补充 <code>Manifest.permission.CAMERA</code> 申请
VUL-011	picUrl TOCTOU	成员变量在多次上传间被覆盖	使用局部变量或 final ID 关联请求
VUL-012	错误信息泄露	服务端 <code>msg</code> 直接 Toast	客户端做错误码映射，过滤敏感信息
VUL-013	缓存未清理	Luban 压缩输出长期残留	上传成功后立即 <code>file.delete()</code>

VUL-014 ~ VUL-016（低危问题）

编号	名称	修复建议
VUL-014	硬编码 requestCode 257	提取为 <code>private static final int REQUEST_CODE_CAMERA = 257;</code>
VUL-015	日志 Tag zq 个人化	统一 <code>LogUtil.TAG</code> 规范
VUL-016	未加固	见 VUL-008

六、攻击链演示 (Kill Chain)

将多个漏洞组合，演示真实攻击场景。

攻击链 A：批量制造已认证账号

- [1] 利用 VUL-002（无签名）
| 接码平台批量注册账号
v
- [2] 利用 VUL-001（相册绕过）
| 准备 100 张爬取的人脸图
v
- [3] 利用 VUL-008（无混淆）
| Frida 脚本自动调用 `uploadPic + realPersonAuth`
v
- [4] 24 小时内产出 1000+ 已认证账号
v
- [5] 流入黑产市场（杀猪盘 / 诈骗 / 引流）

攻击链 B：用户隐私爬取

- [1] 利用 VUL-003（无 SSL Pinning）
| mitmproxy 抓包获取 OSS 直链
v
- [2] 分析 URL 模式（如顺序 ID / 弱 hash）
v
- [3] 遍历下载所有用户的认证照
v
- [4] 涉及《刑法》253条 - 侵犯公民个人信息罪

攻击链 C：认证绕过 + 接管

- [1] 利用 VUL-004 (Hook 篡改)
 | Frida 替换 picUrl 为已认证用户图片
 v
- [2] 服务端若仅校验 URL 合法性而不校验所属用户
 v
- [3] 绕过认证 / 冒充他人身份

七、合规风险评估

7.1 法律法规对照

法规	条款	违规情况
《个人信息保护法》(PIPL)	第 28 条 敏感个人信息处理	未单独取得用户同意
《数据安全法》	第 27 条 数据传输安全	HTTPS 未做 Pinning
《网络安全法》	第 24 条 实名制	实名制可被绕过
《人脸识别技术应用安全管理规定》	第 10 条 活体检测	未实现活体检测
《App 违法违规收集使用个人信息认定方法》	第 4 条	隐私政策未明确收集人脸
GB/T 35273-2020	第 5 条 个人敏感信息	收集流程不规范

7.2 监管处罚风险预估

- **网信办约谈 / 下架**（参考：豆瓣、知乎、小红书 历史处罚）
- **工信部通报**（每月通报名单）
- **罚款上限**：5000 万元 或 上一年度营业额 5%（PIPL 第 66 条）
- **个人责任**：直接负责的主管人员 10万-100万 罚款

八、修复路线图 (Roadmap)

P0（7 天内）

- [] VUL-001：移除相册选项，强制活体检测

- ☐ VUL-002: 上传接口加签名 + 时间戳 + nonce
- ☐ VUL-003: 实现 SSL Pinning
- ☐ VUL-004: 认证逻辑迁移到服务端
- ☐ VUL-005: Release 关闭所有 Log

P1 (30 天内)

- ☐ VUL-006: 收紧 FileProvider 配置
- ☐ VUL-007: 文件类型/大小/名称校验
- ☐ VUL-008: 启用 R8 + 商业加固
- ☐ VUL-009: Activity 生命周期保护

P2 (季度内)

- ☐ VUL-010 ~ VUL-013: 中危问题
- ☐ 隐私协议补充人脸数据告知
- ☐ 上线 SRC 漏洞响应平台
- ☐ 引入第三方安全测试

九、附录

9.1 工具与版本

```
JADX 1.4.7
apktool 2.9.3
Frida 16.1.10
objection 1.11.0
HTTP Toolkit 1.18
MobSF 3.9.7
Burp Suite Pro 2024.x
```

9.2 测试环境

```
设备1: Google Pixel 6 (Android 13, Magisk 已 Root)
设备2: 雷电模拟器 9.0 (Android 9)
代理: mitmproxy 10.x @ 192.168.1.100:8888
```

9.3 时间线

日期	事件
2026-05-10	开始分析
2026-05-12	发现 VUL-001
2026-05-13	完成攻击链验证
2026-05-15	提交报告
待跟进	厂商修复 / 复测

9.4 漏洞披露原则

本报告遵循 **负责任披露 (Responsible Disclosure)**:

- 1. 第 0 天: 私下告知厂商
- 2. 第 30 天: 协助修复, 提供技术支持
- 3. 第 90 天: 如未修复, 向 CNVD/CNNVD 同步
- 4. 第 180 天: 可考虑技术博客脱敏分享 (不含 PoC)

9.5 致谢与免责

本报告所有测试均在合法授权范围内进行, 仅用于提升应用安全性。报告内容**禁止用于任何非法用途**。如有疑问请联系: [联系方式]。

研究员承诺: - 未将测试发现的任何用户数据外传或保留 - 未利用漏洞牟利 - 测试期间未对生产服务造成实质影响

十、参考资料

- OWASP Mobile Application Security Verification Standard (MASVS) v2.0
- OWASP Mobile Security Testing Guide (MSTG)
- 中华人民共和国个人信息保护法 (2021)
- 中华人民共和国数据安全法 (2021)
- 中华人民共和国网络安全法 (2017)
- GB/T 35273-2020 信息安全技术 个人信息安全规范
- 人脸识别技术应用安全管理规定 (试行) (2023)
- CWE - Common Weakness Enumeration

- CVSS v3.1 Specification
-

报告结束

本文档由 Kiro 协助整理，作为安全研究教学示例使用。